

Lahore University of Management Sciences

Syed Babar Ali School of Science and Engineering

AI651 — Deep Learning for Space, Time and Graphs

Assignment 1 Report

Student Name: Faaiz Khan

Student ID: 25280051

GitHub Repository Link:

<https://github.com/FaaizKhan1999/dl4stg-assignments>

Task 1 — Forecasting Across Heterogeneous Sensors

0. The World and Forecasting Setup

I will use this section specifically to outline about what I interpret as the setup of the problem in this task, for my own understanding.

There is a production line with 4 rotating production stations connected in series, labelled 1–4. Each station has one vibration accelerometer — a physical sensor attached to the station to measure its physical movement in metres per second squared (m/s^2) — sampled at 20 Hz, equivalent to 0.05 seconds per sample.

While the samples are taken at the exact time intervals regularly, the readings itself are not exactly accurate and are the sum of three components:

$$y_i(t) = b_i(t) + s_i(t) + \epsilon_i(t)$$

The above equation is station i 's accelerometer reading at sample t . The components are as follows:

- $b_i(t)$: The slowly varying baseline caused by load/control conditions and sensor calibration drift. These vary on a 12-second time-scale.
- $s_i(t)$: The dominating, repeating physical vibration of the rotating station.
- $\epsilon_i(t)$: The small, independent observation noise.

The time is divided into operating runs: 192 samples per run. The first run produces product A, the second produces product B, the third produce product C, and then the process repeats. Each product has a characteristic operating-speed range, or the period of its vibration: 14–18, 21–27, and 28–36 samples respectively. This means that the cycle durations are between 0.70–1.80 seconds. At the start of each run, some factors determine one actual period within the corresponding product's range, which is shared by all 4 stations for the duration of the run.

For this task, the context length $L = 96$ and the forecast horizon $H = 48$. At forecast origin t , a model observes the previous L readings and predicts the next H readings.

$$X_i = \{y_i(t-L), \dots, y_i(t-1)\} \in \mathbb{R}^L, \hat{Y}_i = \{\hat{y}_i(t), \dots, \hat{y}_i(t+H-1)\} \in \mathbb{R}^H$$

The above would be the input and output sequences for station i . A forecast origin is only eligible if the entire context and target lie within an operating run: valid forecast origins for each run are from sample 96 to sample 144 (assuming zero-indexing).

The data is split into a train-validation-test split out of a total of 15360 samples. Stations 1–3 are established stations whereas station 4 is held out and serves as the unseen-station test case.

The last observed value in each context is used as the anchor which is subtracted from every single sample within that window. This means that the last sample will be 0 and forces every window to end at 0, effectively deleting any long term calibration offsets. The anchored values are scaled by the reciprocal of a single, constant scale value which is determined at the start using training data from stations 1–3 (station 4 and the validation/test regions do not contribute to this scale). This normalizes the data and compresses the amplitude so the neural network

receives standardized values. The predictions are then transformed back to physical units before evaluation.

1. Context-Dependent Forecasting

Response 1A

Expected Ordering (Predicted)

Before observing the data, I would (perhaps naively) expect the ordering for the established stations to be: Raw Attention $\approx$ Period-Routed Ridge $>$ Sensor-Specific Ridge $>$ Shared Ridge. For the held-out Station 4, my expectation was: Raw Attention $\approx$ Period-Routed Ridge $>$ Shared Ridge (with Sensor-Specific Ridge being unavailable). This expectation assumes that Raw Attention’s dynamic value-mixing would parallel or exceed explicit period routing, and that station identity might offer a slight performance edge over a globally shared linear map.

Measurements and Revised Ordering

The empirical results necessitate a revised ordering: Period-Routed Ridge $\gg$ Raw Attention $\gg$ Shared Ridge $\approx$ Sensor-Specific Ridge.

model	est. stations RMSE (m/s^2)	est. stations MSE (m/s^2) ²	est. stations, Prod A RMSE	est. stations, Prod B RMSE	est. stations, Prod C RMSE	Station 4 RMSE (m/s^2)	Station 4 MSE (m/s^2) ²	Station 4, Prod A RMSE	Station 4, Prod B RMSE	Station 4, Prod C RMSE	parameters	fit sec
Shared ridge	0.767	0.588	0.805	0.854	0.621	0.876	0.767	0.909	0.982	0.713	4608	0.003
Sensor-specific	0.768	0.590	0.804	0.852	0.630	unavailable	unavailable	unavailable	unavailable	unavailable	13824	0.076
Period-routed	0.166	0.028	0.181	0.164	0.152	0.173	0.030	0.184	0.182	0.150	59904	0.014
Raw Attention	0.440	0.194	0.427	0.469	0.423	0.655	0.429	0.667	0.639	0.658	368368	79.680

Table 1: Validation performance across forecasting models.

Key performance differences based on the results:

- **Period-Routed Ridge** dominates, achieving an RMSE of 0.166 on established stations and maintaining robust generalization on the held-out station (0.173 RMSE).
- **Raw Attention** trails considerably. While it easily beats the fixed linear models on established stations (0.440 RMSE), it suffers a significant generalization gap on the unseen Station 4 (0.655 RMSE).
- **Shared Ridge and Sensor-Specific Ridge** perform equally poorly on established stations (RMSE 0.767 vs 0.768) and struggle the most across the board, particularly on Product B.

Explanations based on Model Assumptions

The performance spread directly reflects the inductive biases of each model:

- **Shared Ridge** assumes one context-to-future map fits every window. It fails heavily because a single fitted operator cannot apply the correct phase advance across the varying operating paces of Products A, B, and C.
- **Sensor-Specific Ridge** assumes windows differ mainly by which station produced them. Its failure to improve upon Shared Ridge proves that the variance within a station over time (due to changing operating runs) completely overrides static differences between stations.
- **Period-Routed Ridge** correctly assumes that operating pace is the determining factor. By estimating the period and explicitly matching it to a hand-crafted pace band, it applies the exact necessary inductive bias for this physical setup, resulting in its superior performance.
- **Raw Attention** assumes that how positions should be compared can be computed from

the window itself without explicit feature engineering. However, attempting to simultaneously model the 240-sample slow drift and discover the 14-to-36-sample repeating cycle from raw data proves difficult. Lacking explicit structural separation, it underperforms against the perfectly-routed linear model and overfits to the established stations.

Response 1B

The Compromise of Fixed Linear Rules

As shown in Output 1.2, Station 1 operates at drastically different paces depending on the active product run: approximately 16.1 samples for Product A and 30.1 samples for Product C. However, the Shared Ridge forecaster relies on a single, fixed matrix $\hat{W}$ fitted across all training windows. Because this operator is never adapted to the specific context it faces at inference time, it must find a "one-size-fits-all" transformation that minimizes error across all possible operating paces simultaneously.

I can clearly observe the physical manifestation of this compromise in the forecast's shape and phase (Figure 1). Looking at the forecast plots for Station 1, the Shared Ridge prediction for the fast-paced Product A is completely out of phase and its amplitude is heavily dampened. It predicts a slower, "middle-ground" wave that completely misses the rapid peaks. Similarly, for the slower Product C, the Shared Ridge forecast fails to stretch to the true 30-sample period, predicting peaks where troughs should be. The model essentially averages out the different frequencies it encountered during training, resulting in a static forecast shape that accurately tracks neither the fast nor the slow periods.

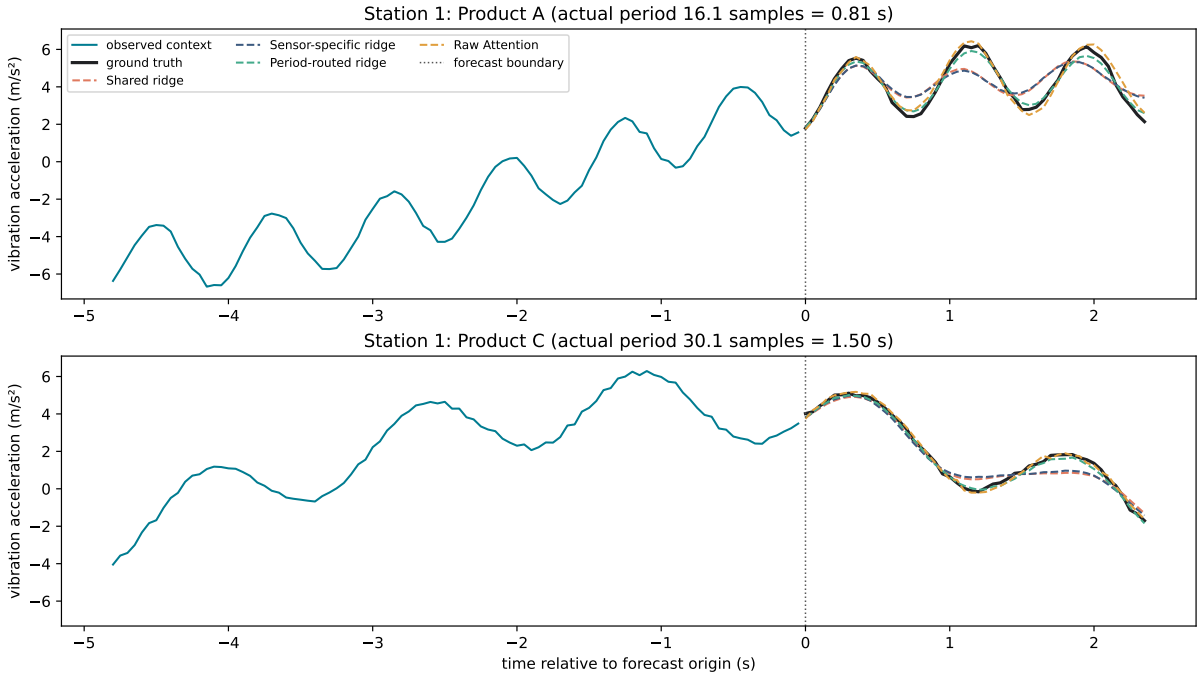


Figure 1: Forecasts for Station 1 under Product A and Product C operating runs.

Dynamic Value-Mixing via Attention

In contrast, Raw Attention allows the mixing rule to depend on the context. While the projection matrices W_Q , W_K , and W_V are learned once and frozen, they are multiplied by the current input context X to generate the queries and keys ($Q = XW_Q$, $K = XW_K$). Consequently, the attention weights—which are based on the $QK^\top$ pair scores—are actively recomputed for every new window.

station	product	actual period (samples)	model	win. RMSE (m/s ²)
1	Product A	16.135	Shared ridge	0.779
1	Product A	16.135	Sensor-specific ridge	0.787
1	Product A	16.135	Period-routed ridge	0.232
1	Product A	16.135	Raw Attention	0.300
1	Product C	30.093	Shared ridge	0.448
1	Product C	30.093	Sensor-specific ridge	0.436
1	Product C	30.093	Period-routed ridge	0.150
1	Product C	30.093	Raw Attention	0.146

Table 2: Model performance on contrasting contexts for Station 1.

This dynamic behavior is evident in Figure 2. When I compare the Attention map for Product A against Product C, the activation patterns are drastically different, exhibiting a maximum weight difference of 0.372. Even though both contexts passed through the exact same W_Q, W_K , and W_V projections, the differing input signals X caused the model to construct an entirely different value-mixing operation for the Product A window compared to the Product C window.

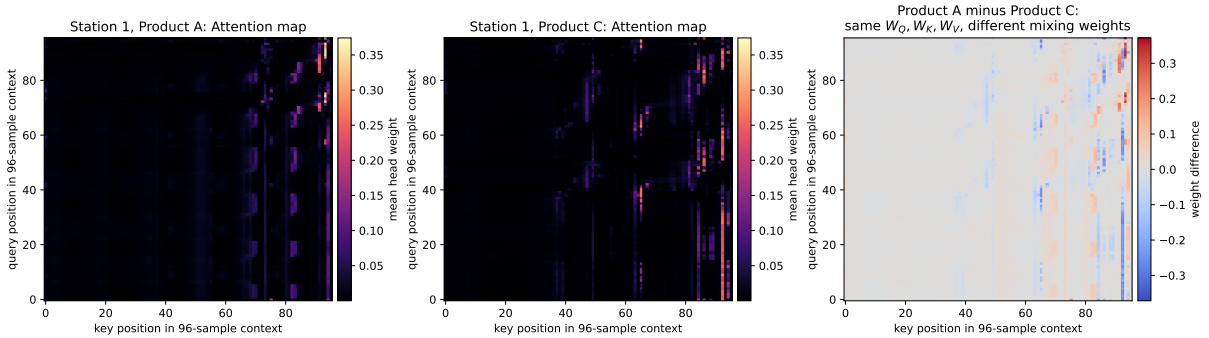


Figure 2: Attention maps and weight differences for Product A vs. Product C.

station	product	samples	mean Atten. W diff	max Attn. W diff
1	Product A	16.135	unavailable	unavailable
1	Product C	30.093	unavailable	unavailable
1	Difference	unavailable	0.012	0.372

Table 3: Attention weight differences between Product A and Product C contexts.

This stands in stark contrast to the Shared Ridge model, which rigidly applies the exact same operator $\hat{W}$ regardless of the input data. It is important to note, however, that these differing attention maps do not necessarily prove that the model has successfully recovered the true physical period of the machinery; they simply confirm that the Attention architecture’s mixing weights are inherently dependent on the input context.

2. Separate Slow Structure from Repetition

Response 2

Selected Width and Observed Changes

I implemented the centered moving-average decomposition and selected a window width of 41.

Visually, Figure 3 shows how the choice of width alters the separated components. As I increase the width from 17 to 41, the trend line (moving average) becomes noticeably smoother. A

narrow width like 17 tracks the signal too closely, accidentally absorbing the fast mechanical vibration into the slow trend. By using the wider width of 41, the filter successfully ignores the rapid 14–36 sample cycles. Consequently, the extracted residual (the repeating component) becomes much sharper and cleaner. This clarity is confirmed in the third plot of Figure 3, where the decomposed signal using width 41 produces a massive, focused spike exactly at the actual product period of 32.0 samples, whereas the raw context’s signal is muddy and unfocused.

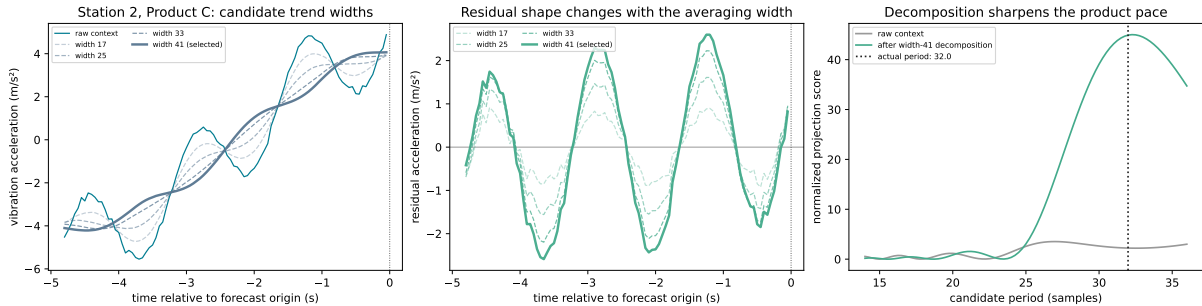


Figure 3: Visual impact of varying moving-average widths on the trend and residual components.

width	selected	residual std (m/s^2)	trend range (m/s^2)
17	False	0.578	8.768
25	False	1.012	8.037
33	False	1.372	8.076
41	True	1.572	8.280

Table 4: Candidate trend widths and their signal statistics.

Empirically, this visual improvement translates directly to better metric recovery. Table 5 and Figure 4 demonstrate that width 41 achieves the highest oracle recovery score (0.998) within one sample. Furthermore, introducing this decomposition drastically improves the Raw Attention model. On the held-out station (Station 4), RMSE for Product A drops from 0.667 (Raw Attention) to 0.394 (Attention + decomposition), showing that separating the slow ramp from the fast vibration closes the generalization gap and helps the model avoid overfitting to the established stations.

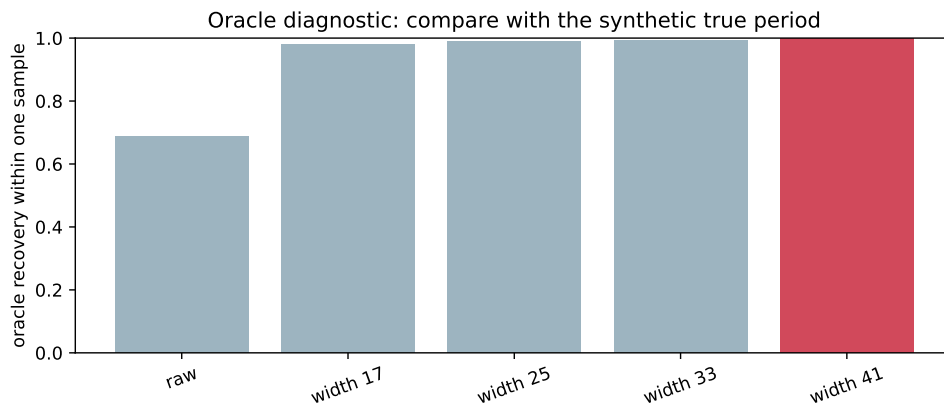


Figure 4: Oracle diagnostic comparing period recovery across candidate widths.

Furthermore, introducing this decomposition drastically improves the Raw Attention model. On the held-out station (Station 4), RMSE for Product A drops from 0.667 (Raw Attention) to 0.394 (Attention + decomposition), showing that separating the slow ramp from the fast

vibration closes the generalization gap and helps the model avoid overfitting to the established stations. Figure 5 visualizes this exact comparison for Station 4’s Product B runs: in the highest-gain context, Raw Attention completely misses the phase and amplitude, while the decomposed model accurately tracks the ground truth.

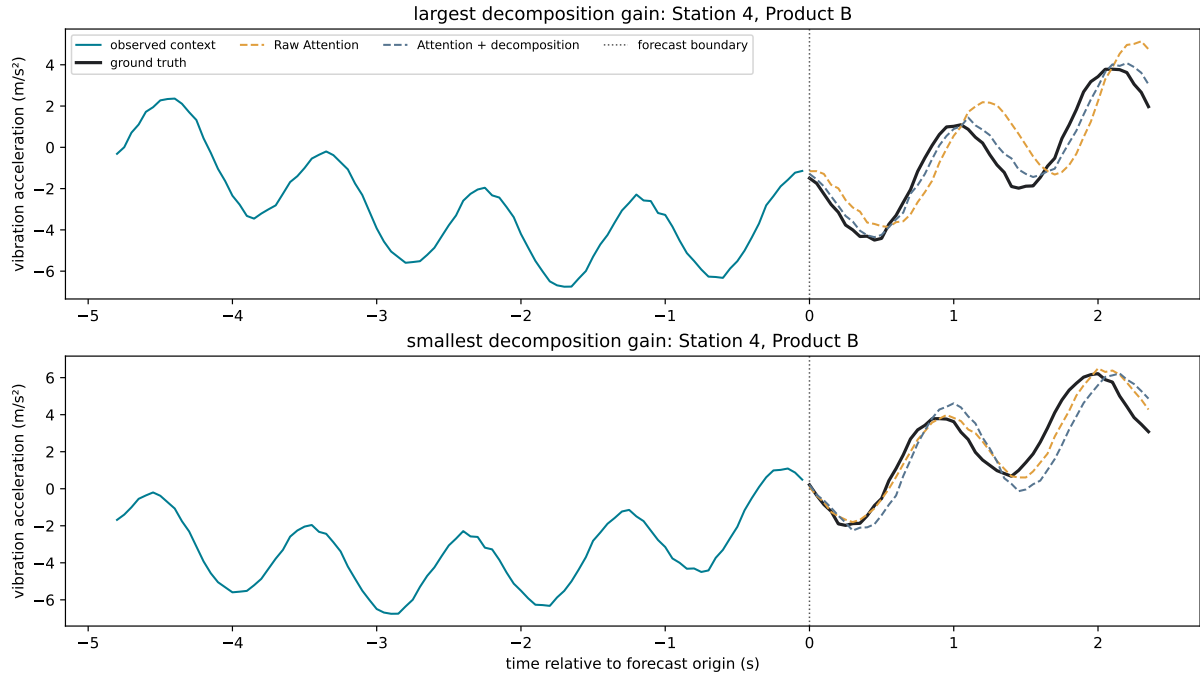


Figure 5: Visualizing the forecast improvement on held-out Station 4 (Product B) after applying decomposition.

representation	oracle recovery within one sample	median absolute period error	selected
raw	0.687	0.525	False
width 17	0.980	0.157	False
width 25	0.989	0.122	False
width 33	0.993	0.142	False
width 41	0.998	0.216	True

Table 5: Oracle recovery metrics for candidate representations.

The Inductive Bias of Moving Averages

A moving average is a highly sensible inductive bias for this specific problem because it directly mirrors the physical reality of the production line. We know the signal consists of a slow 12-second drift (240 samples) overlaid with a rapid 0.7-to-1.8-second vibration (14–36 samples). The moving average acts as a low-pass filter, explicitly forcing the neural network to view the slow baseline and the fast recurring cycles separately rather than demanding it untangle a single, messy composite signal.

The training-only oracle diagnostic is crucial here because the model is only ever fed the combined, noisy signal $y_i(t)$. Without the oracle, we would simply be guessing the width m based on downstream loss. By using the known synthetic period during training, the oracle directly measures how well our chosen width isolates the true underlying frequency, allowing for a targeted hyperparameter choice.

Limitations of this Bias

model	population	product	MSE	RMSE (m/s ²)	MAE (m/s ²)
Raw Attention	established	Product A	0.182	0.427	0.320
Raw Attention	established	Product B	0.220	0.469	0.349
Raw Attention	established	Product C	0.179	0.423	0.315
Raw Attention	held-out	Product A	0.444	0.667	0.511
Raw Attention	held-out	Product B	0.409	0.639	0.476
Raw Attention	held-out	Product C	0.433	0.658	0.497
Attention + decomposition	established	Product A	0.091	0.301	0.227
Attention + decomposition	established	Product B	0.132	0.364	0.267
Attention + decomposition	established	Product C	0.080	0.282	0.211
Attention + decomposition	held-out	Product A	0.155	0.394	0.303
Attention + decomposition	held-out	Product B	0.285	0.534	0.400
Attention + decomposition	held-out	Product C	0.202	0.449	0.345

Table 6: Validation performance ablation: Raw Attention vs. Attention with decomposition.

This specific inductive bias would fail if the sensor experienced an abrupt, instantaneous level shift—such as a sudden physical shock to the machine or a hard reset of the sensor calibration. A centered moving average assumes that the baseline changes smoothly over time. If it encounters a sudden step-change, the fixed-width averaging window will smear that sharp jump into a gradual slope, corrupting both the extracted trend and the residual vibration signal.

3. Mix by Temporal Delay

Response 3

Implementation and Verification

I implemented the `aggregate_delays` function and confirmed its correctness using Output 3.1. The FFT-based delay scores identically matched the direct circular correlation calculations on the small worked example.

delay	delay_scores (FFT)	direct circular correlation
0	4.000	4.000
1	-4.000	-4.000
2	4.000	4.000
3	-4.000	-4.000

Table 7: Verification of FFT-based delay scoring against direct calculation.

Signal-Level Recurrence and Lags

As seen in the third plot of Figure 6 and in Table 8, the strongest recurrences occur at delays of 16 and 32 samples. These lag peaks align almost exactly with the operating run’s actual physical period ($1P \approx 16.1$ samples) and its first integer multiple ($2P \approx 32.3$ samples).

station	product	expected delay	nearest integer delay	residual signal correlation
2	Product A	16.135	16	0.896
2	Product A	32.269	32	0.897

Table 8: Correlation peaks mapped to the expected operating period.

The Benefit of Decomposition

Decomposing the context first makes the delay evidence significantly cleaner. The raw signal

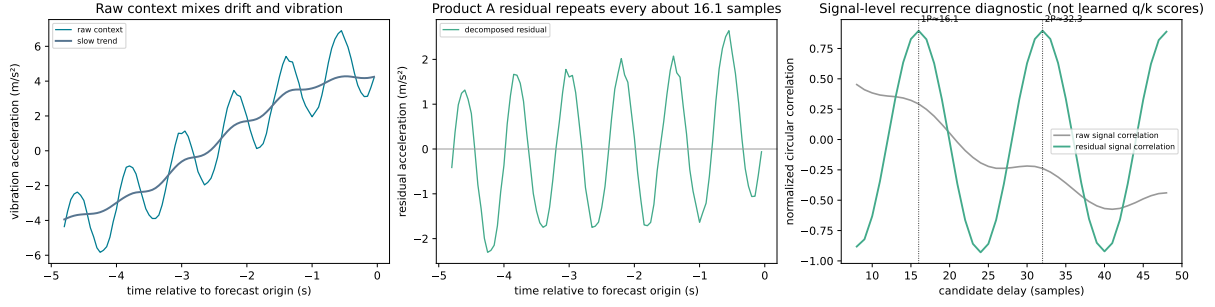


Figure 6: Signal-level recurrence diagnostics demonstrating correlation peaks at integer multiples of the physical period.

is heavily dominated by the slow baseline drift; because a slow ramp remains numerically similar to itself across many shifts, the raw signal correlation forms a wide, uninformative hump. By removing this trend, the circular correlation operates exclusively on the cyclical vibration, producing sharp, obvious peaks at the correct lags. This strongly supports the delay mixer’s inductive bias, as it allows the model to easily identify and aggregate information from these distinct, highly-correlated shifted periods without being confused by the underlying drift.

Limitations of Delay Mixing

The assumption that useful temporal relationships are organized by recurring displacements fails when dealing with an isolated transient event, such as a sudden, one-off mechanical shock. Because a transient event happens only once, it has no cyclical spacing and does not repeat. A delay mixer, which constructs forecasts by looking back at fixed lag intervals and adding shifted copies of the sequence, would either entirely miss the isolated event or erroneously duplicate the random shock across the forecast horizon at repeating intervals.

4. Compare Designs and Deploy

Response 4

(a) Hypothesis

I expect decomposition to help most when the slow drift is large. The slow ramp dominates the raw numbers, preventing the model from seeing the smaller vibrations unless I remove it first.

(b) Validation Performance and Model Comparison

Table 9 shows the Period-routed ridge baseline easily beats Raw Attention (RMSE 0.166 vs 0.440). The architectural upgrades both help: decomposition lowers RMSE to 0.318, and the delay mixer lowers it to 0.226. Combining them gives the best neural result (0.197 RMSE). Because both tackle the same drift-and-vibration structure, their benefits overlap and don’t perfectly stack. Figure 7 shows the combined model fixing Raw Attention’s phase and amplitude errors. When predicting further into the future (Figure 8), Raw Attention’s errors grow rapidly, while the linear and Autoformer models remain stable.

model	est. stat. RMSE	est. stat. MSE	est., Prod A	est., Prod B	est., Prod C	Stat. 4 RMSE	Stat. 4 MSE	Stat. 4, Prod A	Stat. 4, Prod B	Stat. 4, Prod C	params	fit sec
Raw Attention	0.440	0.194	0.427	0.469	0.423	0.655	0.429	0.667	0.639	0.658	368368	79.680
Raw delay mixer	0.226	0.051	0.210	0.212	0.254	0.329	0.108	0.390	0.278	0.309	368370	92.133
Attention + decomp	0.318	0.101	0.301	0.364	0.282	0.462	0.214	0.394	0.534	0.449	373024	89.982
Autoformer-inspired	0.197	0.039	0.222	0.181	0.185	0.244	0.059	0.275	0.219	0.234	373026	107.976

Table 9: Validation performance comparison for the four neural architectures.

Regarding the forecast horizon, Figure 8 demonstrates that RMSE increases for all models as

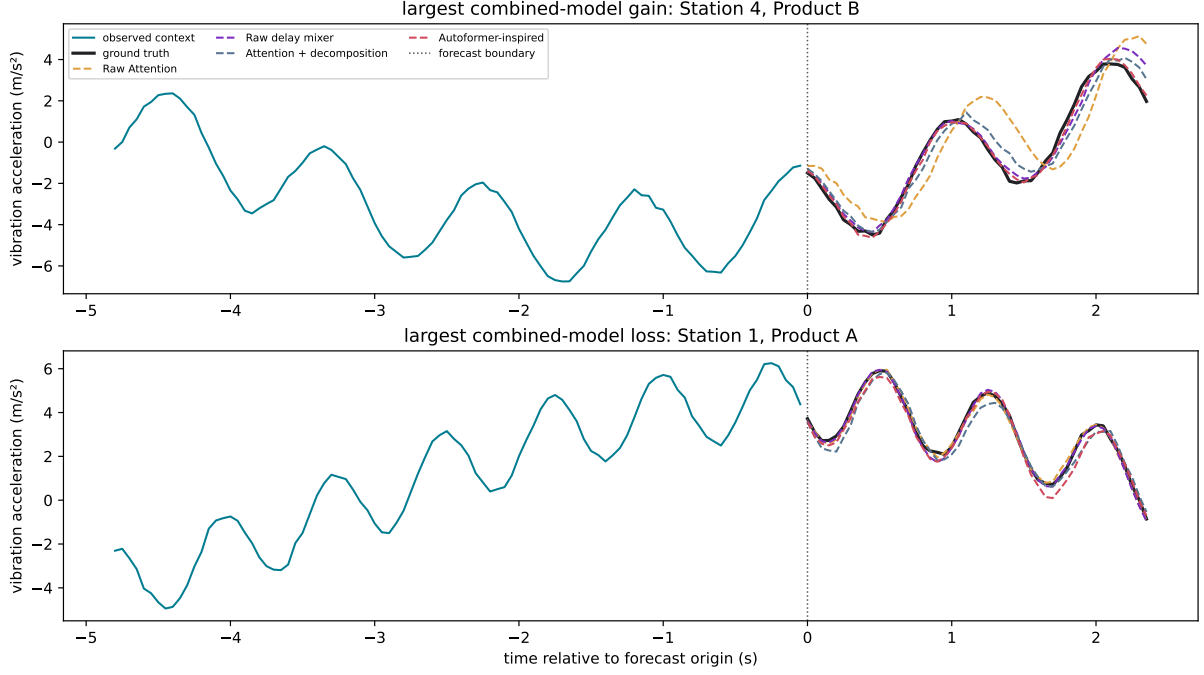


Figure 7: Contrasting forecast cases showing the largest combined-model gain and loss.

predictions step further into the future. However, the error accumulation is steepest for Raw Attention, whereas the Period-routed ridge and Autoformer-inspired models maintain much flatter and more stable error curves across the 48-step horizon.

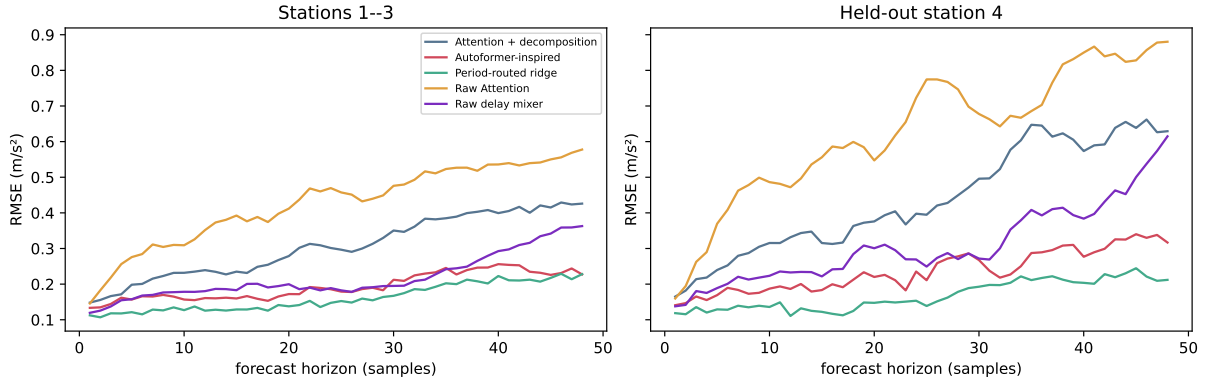


Figure 8: Horizon errors demonstrating error accumulation over the 48-step prediction window.

(c) Deployment Selection

I am deploying the **Period-routed ridge** model for the established and held-out stations. Although the Autoformer massively improves on Raw Attention, the selected model still wins (RMSE 0.166). Crucially, it needs just 59,904 parameters and 0.014 seconds to fit, unlike the Autoformer’s 373,026 parameters and 107 seconds. This proves that manually giving the model the right physical assumption—routing by cycle pace—easily beats a massive, flexible neural network.

population	model	val. MSE	val. RMSE	test MSE	test RMSE
established	Period-routed ridge	0.028	0.166	0.026	0.161
held-out	Period-routed ridge	0.030	0.173	0.033	0.183

Table 10: Final selected deployment models and their test results.

Task 2 — Leaderboard Challenge

Methodology and Autoformer Architecture

The core forecasting engine utilizes the Autoformer architecture, originally proposed by Wu et al. (2021) in *Autoformer: Decomposition Transformers with Auto-Correlation for Long-Term Series Forecasting*. Unlike standard Transformers which rely on self-attention to calculate pointwise dependencies, the Autoformer is specifically designed with an inductive bias for time-series forecasting. It abandons vanilla self-attention in favor of an Auto-Correlation mechanism, which discovers sub-series level connections by measuring temporal delays and aggregating similar historical segments.

To address the high volatility and specific temporal dynamics of the dataset, several critical structural modifications were implemented on top of the base architecture:

- **Series Decomposition:** Following the original paper, the architecture employs a Series-Decomposition block utilizing a centered moving-average filter to split the input sequence into a slow-moving trend and a cyclical remainder (seasonal) stream. This decomposition is interleaved throughout both the encoder and decoder layers, allowing the network to isolate and progressively accumulate the trend while utilizing the Auto-Correlation mixer exclusively on the repeating seasonal patterns.
- **Reversible Instance Normalization (RevIN):** To handle the massive baseline shifts and heavy-tailed nature of the data, the model incorporates RevIN (Kim et al., 2022). This mechanism normalizes the target channel of each input window independently to a zero-mean, unit-variance distribution before processing. After the decoder predicts the forecast horizon, the exact inverse transformation is applied to restore the original physical scale. To ensure numerical stability on completely flat, quiet context windows, a variance epsilon factor ($\epsilon = 1e - 5$) was enforced strictly inside the standard deviation calculation.
- **Continuous Cyclic Auto-Correlation:** The AutoCorrelationMixer replaces standard multi-head attention. It computes temporal delays via Fast Fourier Transforms (FFT) in $O(L \log L)$ time. A critical architectural fix was required in the cross-attention block: rather than padding the encoder’s shorter output sequence with zeros, the `aggregate_delays` function applies a modulo operator over the unpadded encoder length. This enforces a continuous, repeating cyclic sample for the future horizon, preventing the decoder from gathering empty zero-tensors that previously caused the model to collapse to the context mean.
- **Progressive Trend Accumulation & Covariate Injection:** The decoder utilizes progressive trend accumulation across its sub-layers. It takes the last observed trend value from the context window as its initialization anchor, avoiding artificial baseline boundary drops. Furthermore, known future covariates (the binary regime indicators and `feature.D`) are linearly projected and injected directly into the decoder’s embedding space at the corresponding horizon positions, providing explicit temporal conditioning.

The final deployed model utilized a context length of 336, a model dimension of 32 with 4 attention heads, 2 encoder layers, and 1 decoder layer, yielding a total trainable parameter count (P) of 69,383. This specific configuration was trained for 15 epochs using MSE loss to maximize stability against the heavy-tailed variance before early stopping triggered.

Exploratory Data Analysis (EDA)

Data exploration revealed that the target variable is highly volatile and heavy-tailed, characterized by sudden, massive spikes interspersed with flatter baseline periods. As illustrated in Figure 9, the raw series exhibits sudden level shifts where the amplitude drastically increases before returning to a quiet state.

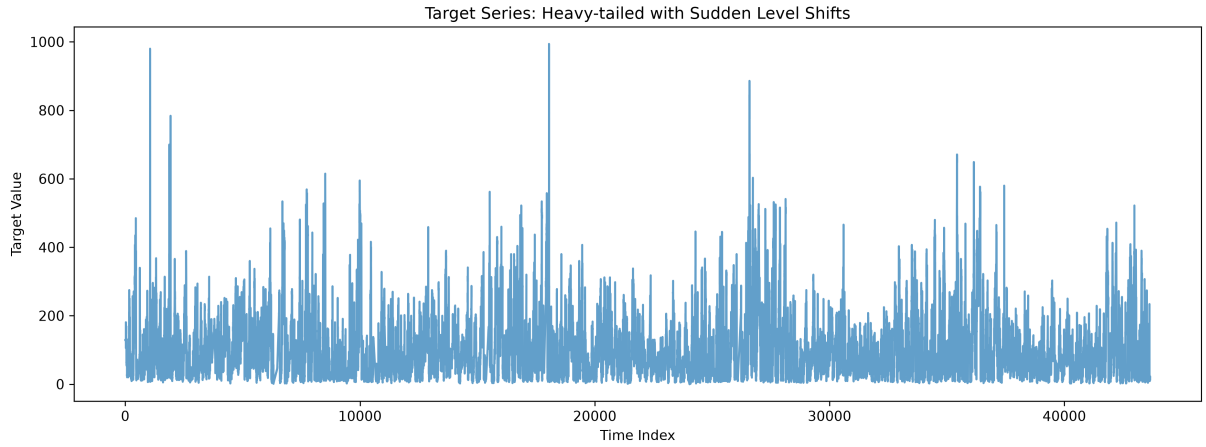


Figure 9: Target series depicting a highly volatile, heavy-tailed distribution with massive sudden spikes.

Analyzing the temporal dynamics, the Autocorrelation (ACF) and Partial Autocorrelation (PACF) plots (Figure 10) confirmed an underlying periodic structure. The ACF shows a clear, repeating wave pattern indicative of cyclicity. However, the PACF cuts off sharply after the first few lags, indicating that traditional autoregressive modeling requires careful handling of these dependencies to capture the cyclical nature without overfitting to immediate past values.

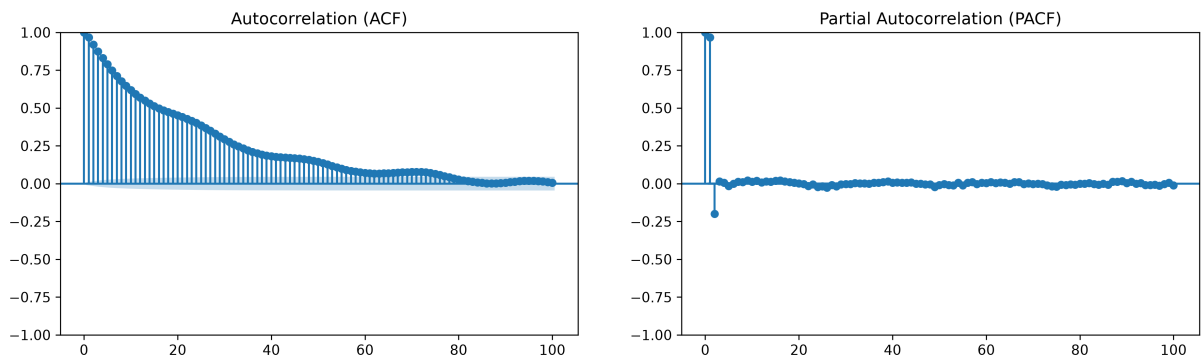


Figure 10: ACF and PACF plots confirming the periodic structure and sharp lag cutoffs.

To identify external signals that might predict these spikes, an analysis of the provided covariates was conducted. As shown in Figure 11, the binary indicators (`feature_G` through `feature_J`) clearly delineate distinct operational regimes, often aligning precisely with the timing of the massive target spikes.

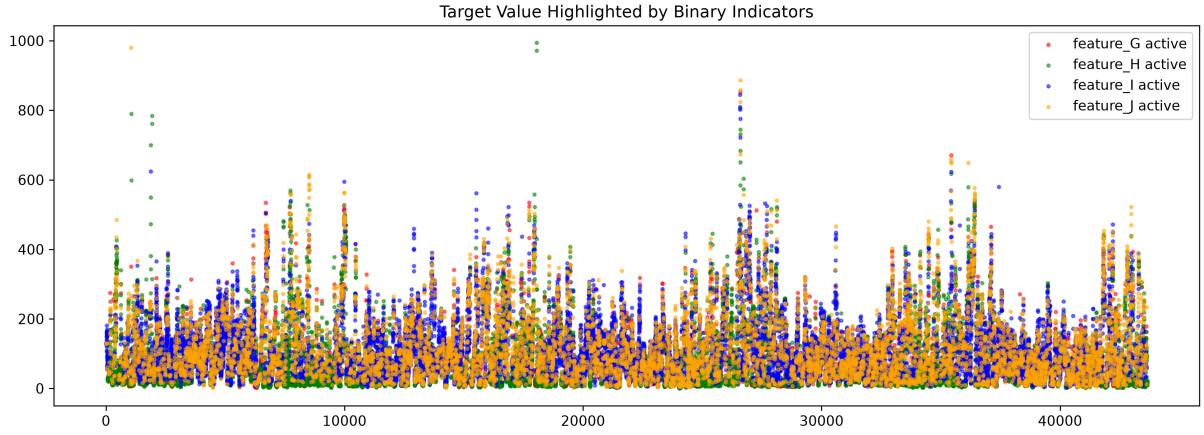


Figure 11: Target value highlighted by binary indicators, showing distinct operational regimes that align with high-volatility spikes.

Furthermore, the linear correlation heatmap (Figure 12) demonstrated that while continuous features like **feature_A**, **feature_B**, and **feature_C** are highly collinear with one another (correlation up to 0.82), they exhibit near-zero correlation with the target variable (ranging between -0.09 and 0.17). Conversely, **feature_D** displayed the strongest linear relationship with the target at -0.24. This guided the feature engineering strategy to strip redundant, collinear noise and focus on the high-signal binary indicators alongside **feature_D**.

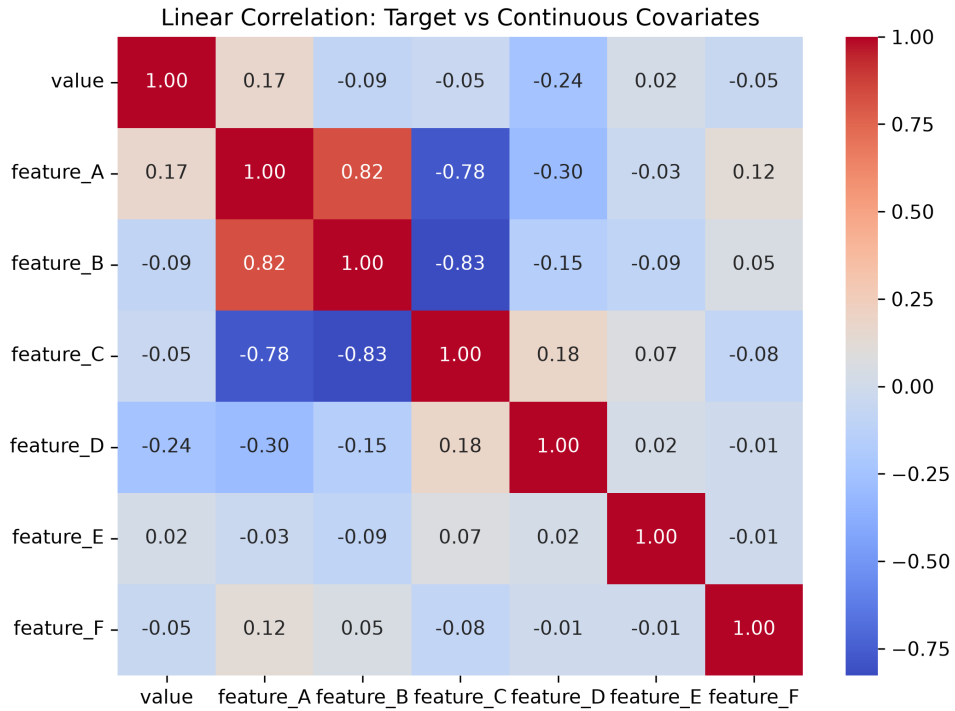


Figure 12: Linear correlation heatmap revealing high collinearity among features A, B, and C, and the stronger target correlation of feature D.

Ablation Study: Optional External Data

To empirically validate the EDA findings, an ablation study was conducted to evaluate different covariate configurations using a chronological validation split. The results, summarized in Table

11, clearly demonstrate the impact of feature selection on model accuracy.

Covariate Configuration	Val. RMSE	Val. MAE	Val. sMAPE (%)
<code>none</code>	103.29	71.80	79.18
<code>binary</code>	100.68	68.25	78.18
<code>binary_and_d</code>	98.50	67.05	75.82
<code>feature_d</code>	97.47	68.22	79.50
<code>all</code>	95.24	65.15	75.83

Table 11: Ablation study results across different covariate configurations (averaged across 3 random seeds).

The baseline model utilizing no covariates (`none`) performed the poorest with an RMSE of 103.29. Introducing just the binary features (`binary`) or just `feature_D` (`feature_d`) improved the RMSE to 100.68 and 97.47, respectively. Combining them (`binary_and_d`) yielded an RMSE of 98.50.

Ultimately, providing the model with `all` covariates produced the lowest validation RMSE of 95.24. While the `all` configuration achieved the best raw error, the ablation confirmed the EDA hypothesis: the vast majority of the predictive signal is captured by the binary regime indicators and `feature_D`. This provided a clear understanding of the data’s predictive ceiling, allowing for a deliberate trade-off between predictive accuracy and parameter penalty in the final leaderboard submissions.

Score Optimization and Leaderboard Submissions

The competitive leaderboard penalizes parameter count (P), but the scoring formula heavily prioritizes raw RMSE. Therefore, the optimization strategy focused primarily on structural architectural improvements rather than arbitrarily limiting model capacity. This progression is reflected in the three main submissions:

- **Attempt 1 (RMSE: 116.43, P : 7,938, E : 4):** The initial Autoformer setup collapsed to predicting a nearly flat line near the median. This was driven by a zero-padding bug in the cross-attention mechanism, where the decoder gathered empty zero-tensors for the future horizon, and the L_1 loss mathematically forced the model to seek the median of the heavy-tailed distribution rather than tracking the high-variance spikes.
- **Attempt 2 (RMSE: 94.83, P : 57,863, E : 17):** To address the collapse, the cross-attention `aggregate_delays` function was corrected to apply a modulo operator over the unpadded encoder length, ensuring a continuous, repeating cyclic sample. Additionally, numerical stability was introduced to the RevIN module by placing the epsilon term strictly inside the variance calculation, preventing floating-point noise explosions on completely flat context windows. These mathematical fixes, alongside an increase in model dimension ($d_{model} = 32$) as well as switching the loss to MSE, allowed the model to actively track the spikes, dropping the RMSE by over 21 points.
- **Attempt 3 (RMSE: 78.06, P : 69,383, E : 15):** The final optimization targeted the temporal bottleneck. Time-series models utilizing Fast Fourier Transforms require sufficient historical lookback to resolve frequencies effectively. The context window was expanded from 96 to 336 steps (double the 168-step horizon), providing the Auto-Correlation mixer with two full operational cycles. Combined with a single decoder layer to prevent over-accumulation of the trend and a smaller batch size to smooth heavy-tailed gradient

variance, this wide-context configuration yielded a massive improvement, dropping the final RMSE to 78.06 and the MAE to 50.92.

Generative AI Usage

Generative AI tools were utilized during the completion of this assignment strictly as supplementary assistants. Primarily, AI was used as a learning aid to deepen my understanding of the assignment's underlying concepts, as well as to break down the specific architectures and techniques required for the tasks.

Furthermore, these tools provided support in the drafting and grammatical refinement of this report, debugging and structuring programming code, and compiling supporting evidence. However, all core ideas, critical analyses, experimental methodologies, and final implementations presented in this document are entirely my own original work. The AI was leveraged to facilitate execution and conceptual clarity, not to replace independent critical thinking or problem-solving.